

NHCX InsurancePlanBundle FHIR Generator (PROBLEM 3)

Open-Source LLM-Driven Insurance Policy PDF → NHCX InsurancePlanBundle Pipeline

What We Built

We designed and implemented a **fully open-source, privacy-preserving, LLM-orchestrated pipeline** that converts insurance policy PDFs into **HL7 FHIR R4 InsurancePlanBundle resources compliant with NHCX (National Health Claims Exchange) standards**.

This is not a flat text-to-JSON conversion.

This is a **structured, dependency-aware, rulebook-driven insurance-to-FHIR transformation framework** powered entirely by local Large Language Models — without using any paid APIs.

Privacy-First Architecture & Data Sovereignty

To ensure the highest standards of data sovereignty and privacy in healthcare, our solution **operates entirely through on-premises microservices**. The architecture runs fully offline and locally, eliminating the need for cloud LLM endpoints or external APIs. This local deployment ensures **zero data leakage** and full health data sovereignty, directly **aligning with the Digital Personal Data Protection (DPDP) Act requirements** for secure and compliant processing. To manage the complexity of multi-page document, we use a pre-processing stage called "Text Distillation". This process uses Docling to extract text and filters out narrative "fluff" into an actionable "Fact Sheet". By distilling data before extraction, the system prevents context overflow and optimizes performance on local hardware like the NVIDIA A6000. This structured orchestration maintains strict NHCX rulebook compliance and ensures UUID linking integrity within a secure, production-ready environment.

Why Our Approach Is Different

Most document-to-JSON tools:

- Perform naive text extraction
- Use single prompt generation
- Produce loosely structured JSON
- Fail when financial logic and structured limits are complex

Our system instead:

- ✓ Uses **LLM-Orchestrated Dynamic Workflows**
- ✓ Performs **Insurance Text Distillation**
- ✓ Applies **NHCX Rulebook-Based Structural Constraints**
- ✓ Enforces Dependency Graph Ordering

- ✓ Assembles True NHCX InsurancePlanBundle (Bundle type: collection)
- ✓ Maintains UUID Linking Integrity
- ✓ Runs Fully Offline and Locally
- ✓ Is 100% Open Source

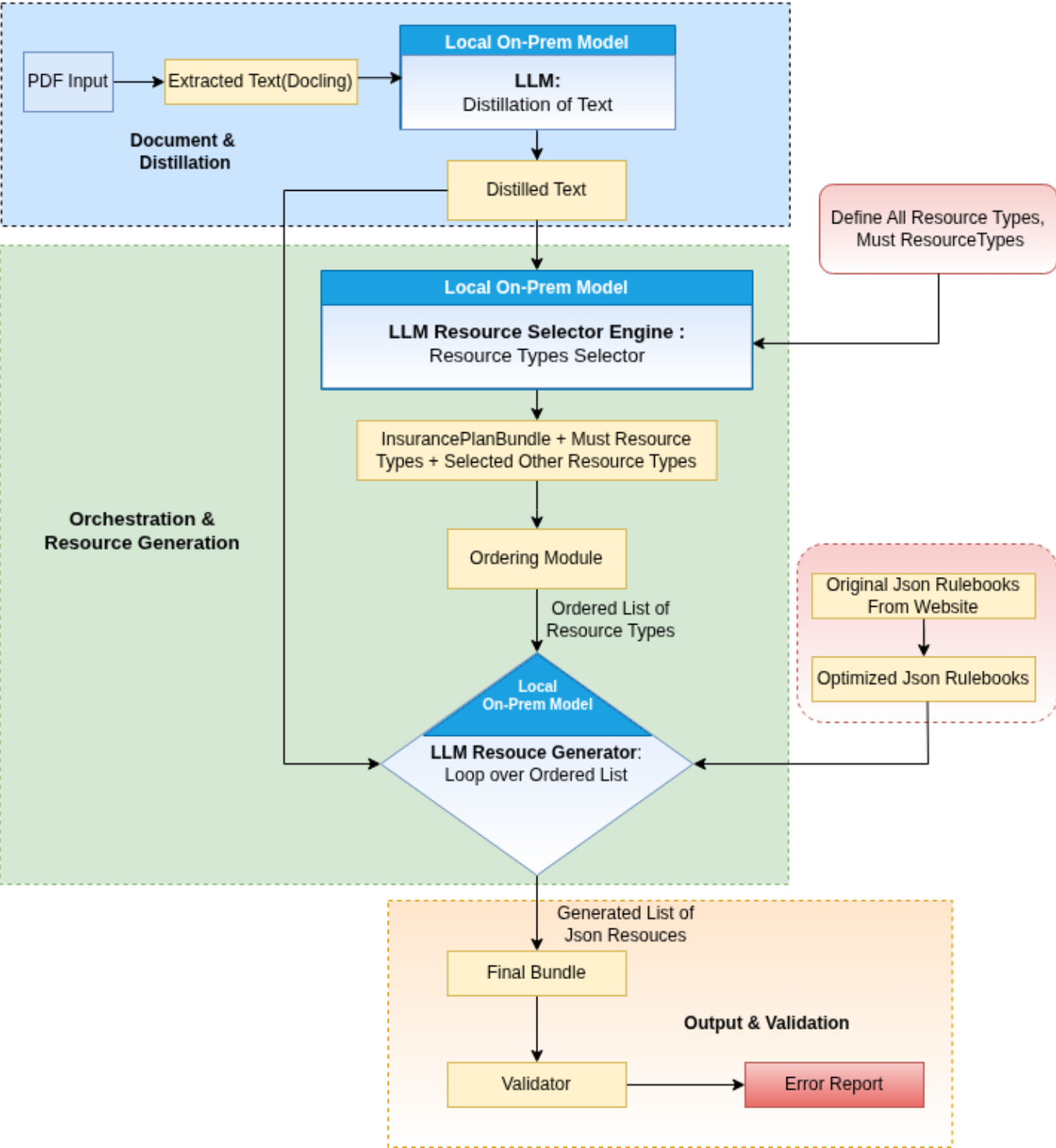
This makes our system architecturally superior to basic extraction-based approaches.



System Architecture

Problem 3

NHCX InsurancePlanBundle FHIR Generator



Core Methodology

Multi-Stage Structured Pipeline

- PDF
- Docling Extraction
 - LLM Distillation of Text

- LLM Resource Selection
- Dynamic Workflow Builder (LangGraph)
- Per-Resource LLM Extraction
- Dependency-Based Linking
- InsurancePlanBundle Assembly
- Final JSON Output

Each resource is generated independently with:

- Dedicated rulebook
- Strict structural prompt
- UUID enforcement
- Financial data preservation rules

This modular architecture significantly improves structural reliability.

2 Insurance Text Distillation

Insurance documents contain:

- Legal preambles
- Financial limits
- Co-pay percentages
- Waiting periods
- Exclusion clauses
- Benefit tables

Instead of feeding the entire raw text directly to the extraction agent, we:

- ✓ Split into chunks
- ✓ Use an LLM Underwriter-style “Fact Sheet” distillation
- ✓ Reconstruct tables where applicable
- ✓ Remove narrative fluff
- ✓ Keep only actionable insurance data

This prevents context overflow and improves extraction quality.

3 Rulebook-Driven Extraction (NHCX Profiles)

Each resource uses its updated StructureDefinition rulebook.

This ensures:

- Proper InsurancePlan modeling
- IRDAI exclusion handling
- SNOMED CT for conditions when required
- Correct Bundle type: collection
- Mandatory profile enforcement
- Strict UUID linking

This is not generic FHIR — it is NHCX-aligned FHIR.

4 Dependency-Aware Dynamic Workflow

We dynamically build a LangGraph workflow based on:

- Mandatory resources
- LLM-selected optional resources
- Predefined dependency graph

Example dependencies:

- InsurancePlan depends on Organization
- DocumentReference depends on Binary
- Bundle assembled last

This deterministic orchestration improves structural correctness.



LLM Strategy

Model Used:

Qwen 2.5 – 32B

Running locally on:

- NVIDIA A6000
- NVIDIA A6000 ADA

Why?

Because:

- Hackathon time constraints
- Limited GPU access
- Requirement for full local deployment
- No external API usage

Even though Qwen 2.5 32B is mid-tier, our architecture compensates via structured orchestration.

With access to:

- 70B+ parameter models
- H100/H200 class GPUs

Accuracy would significantly increase.

For this problem:

Accuracy > Latency

We intentionally prioritized structural accuracy over execution speed.

Open Source Advantage

We did not use:

- OpenAI API
- Paid inference APIs
- Cloud LLM endpoints
- External data services

Everything runs locally.

Benefits:

- ✓ Zero data leakage risk
- ✓ Full insurance data sovereignty
- ✓ Deployable inside insurer infrastructure
- ✓ No compliance concerns
- ✓ Fully open-source stack

This is production-ready from a privacy standpoint.

Performance Observations

For a typical multi-page insurance policy:

GPU	Average Time
NVIDIA A6000	6 – 8 minutes
NVIDIA A6000 ADA	5 – 7 minutes

Stronger GPUs (H100/H200) would reduce processing time significantly.

Latency scales with:

- Policy complexity
- Number of benefit tables
- Number of extracted resources

Microservice Deployment

Live Platform:

 <https://nhcxhackathon.tanuh.ai/>

Features:

- Upload Insurance Policy PDF
- Generate NHCX InsurancePlanBundle
- Download JSON
- Validate Bundle
- View Error Reports

Supports:

- Problem Statement 2 (Clinical Bundles)
- Problem Statement 3 (Insurance Plan Bundle)

Fully open-source backend.



Current Validation Capability

Integrated:

- ✓ HL7 FHIR Validator CLI
- ✓ Error extraction and filtering
- ✓ UI error reporting

However, due to hackathon time constraints:

We could not fully implement automated error correction agents.



Our Big Future Vision: LLM-In-The-Loop Validation

We propose a multi-layer post-processing validation architecture:

Layer 1 – Missing Data Completion

LLM compares:

- Distilled insurance text
- Generated JSON

If policy limits or exclusions exist in text but are missing in JSON, regenerate missing structures.

Layer 2 – Terminology & Coding Validator

LLM verifies:

- IRDAI Exclusion codes
- SNOMED mappings
- Bundle profile conformance

- Financial limit placement accuracy

Corrects structural and coding inconsistencies.

Layer 3 – Iterative FHIR Structural Correction Loop

Loop K times (k = 3 or 4):

1. Generate JSON
2. Run FHIR Validator
3. Feed validation errors to LLM
4. LLM fixes JSON
5. Repeat

This creates a self-improving FHIR refinement loop.

We could not complete this layer due to time constraints.

But architecturally, this dramatically improves bundle accuracy.



Why Our Architecture Is Strong

Insurance policies are harder than clinical notes because:

- Complex financial rules
- Multi-level benefit structures
- Waiting period logic
- Exclusion mappings
- Legal formatting
- Large textual size

Our pipeline handles this via:

- Chunk-based distillation
- Deterministic workflow
- Resource decomposition
- Structured prompts
- Assembly-first logic

This is not extraction.

This is insurance data engineering.



Known Limitations

- Multiple LLM calls increase latency
- No automatic validation correction layer yet

- 32B model requires high memory
- Very large policies stress context windows



Primary Artifact

InsurancePlanBundle
(Bundle type: collection)



Execution Flow

main()
→ get_nhcx_json()
→ extract_distilled_text_from_nhcx_pdf()
→ distill_insurance_text()
→ select_nhcx_resources()
→ build_insurance_workflow()
→ run_extraction_agent()
→ insurance_assembly_node()
→ Final Bundle Output



System Architecture

PDF Input
→ Docling Extraction
→ LLM Distillation
→ Resource Selection
→ Dependency Graph Builder
→ Resource Nodes
→ InsurancePlan Anchor
→ Supporting Resources
→ Attachment Resources
→ Final NHCX Bundle



Why This Is Production-Ready

- Deterministic workflow orchestration
- Rulebook enforcement
- Strict UUID linking
- Local deployment
- Validator integration

- Microservice-ready backend

This system can scale with better models and hardware.

Setup Instructions

Clone Repository

```
git clone https://github.com/tanuh-bcd/NHCX_HACKATHON
cd NHCX_HACKATHON/ocr_service_problem_3/app
```

Create Virtual Environment

```
python -m venv venv
source venv/bin/activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Install Ollama

<https://ollama.com>

```
ollama pull qwen2.5:32b
ollama serve
# You have to modify the llm initialization in llm_requirements.py from
ChatOpenAI to ChatOllama
```

Run Pipeline (CLI)

```
python main.py input.pdf
```

Run as Microservice

```
uvicorn main:app --reload
```



System Requirements

- Python 3.10+
 - 30–32GB RAM recommended
 - GPU strongly recommended
 - NVIDIA A6000 class minimum for 32B model
-



Conclusion

We built:

- A structured LLM-orchestrated NHCX InsurancePlanBundle generator
- A dependency-aware financial FHIR pipeline
- A privacy-preserving open-source framework
- A deployable microservice
- A validation-ready architecture
- A future-ready LLM-in-the-loop correction system

With stronger GPUs and larger LLMs, this can evolve into:

A fully autonomous Insurance Policy → NHCX FHIR transformation engine with near-perfect structural compliance.